

Information Retrieval: Wild Cards e Spelling Correction

1 Wild Cards

Le wild card rappresentano un carattere jolly che può essere utilizzato per rappresentare uno o più caratteri in una stringa.

Ad esempio, la query **mon*** significa "trova tutti i documenti che contengono parole che iniziano con 'mon'". In questo caso, la query restituirà risultati relativi a "monkey", "monitor", "monument", ecc. Risolvere una query di questo tipo è **semplice se il dizionario è codificato tramite un B-tree**: dal momento che le parole sono in ordine lessicografico, è sufficiente prendere le parole nel range $mon \leq w < moo$.

Se invece dovessimo risolvere la query ***mon** diventa più complesso in quanto il dizionario non è ordinato in base al suffisso. La soluzione è semplicemente mantenere quindi un ulteriore B-tree con le parole scritte al contrario: common diventa nommoc, demon diventa nomed, ecc. In questo modo, per risolvere la query ***mon**, è sufficiente cercare le parole che iniziano con nom. Quindi, **mantenendo una copia del dizionario (relativamente costoso), è possibile risolvere facilmente anche le query con wild card iniziali**.

Tuttavia, anche se siamo riusciti a enumerare tutti i termini del dizionario che soddisfano le wildcard, c'è un costo significativo da pagare per recuperare le posting di ciascun termine enumerato dal momento che è necessario fare l'unione delle posting list, e quindi restituisce moltissimi documenti. Sapendo che gli utenti tendono a digitare query più complesse (es. **de* AND *mon**) ciò può portare all'esecuzione di operazioni molto costose.

E per queries del tipo **co*tion**? Un'idea naive potrebbe essere: recupera **co***, ***tion** e poi fai l'intersezione. Tuttavia ciò è estremamente costoso per le ragioni spiegate prima, per questo si usa un'approccio più efficiente: i **permuterm index**.

1.1 Permuterm Index

L'idea dietro i permuterm è quella di **trasformare ogni wildcard query in una query con * alla fine**, di modo che basti una normale ricerca per prefisso nel B-tree. Per fare ciò:

1. Si aggiunge un simbolo speciale \$ alla fine di ogni termine del dizionario (es. hello diventa hello\$).
2. Si generano tutte le rotazioni del termine ottenuto (es. hello\$ diventa hello\$, ello\$h, llo\$he, lo\$hel, o\$hell, \$hello).
3. Si aggiungono tutte le rotazioni all'indice del dizionario, ma puntano tutte allo stesso termine originale (nel caso dell'esempio hello).

Vediamo ora come funziona, con le varie tipologie di query:

- **Query senza wildcard, X**: si cerca semplicemente X\$. Ex. hello, quindi cerco hello\$ che punta alla parola esatta.
- **Query con wildcard finale, X***: si cerca X\$*. Ex. voglio cercare hel*, quindi cerco \$hel*. Questo perché ogni parola che inizia per hel avrà tra le sue rotazioni \$hel, e quindi cercando \$hel* ottengo tutte le parole che iniziano per hel (nota come sto sempre sfruttando l'asterisco alla fine della query).
- **Query con wildcard iniziale, *X**: si cerca X\$*. Ex. voglio cercare *llo, quindi cerco llo\$*. Questo perché ogni parola che finisce per llo ha tra le sue rotazioni llo\$, e quindi cercando llo\$* punto a tutte le parole che finiscono per llo.

- **Query con wildcard ai due lati, $*X*$:** si cerca $X*$. Ex. voglio cercare $*ell*$, quindi tutti i termini che contengono ell da qualche parte. Se una rotazione comincia per ell, vuol dire che nel termine originale ell appare come sottostringa! (per hello ello\$h), per cui cerco $ell*$ e ottengo tutte le parole che contengono ell da qualche parte.
- **Query con wildcard in mezzo, $X*Y$:** si cerca $Y\$X*$. Ex. voglio cercare $h*lo$, quindi cerco $lo\$h*$. Questo perché la query $h*lo$ vuol dire che la parola deve iniziare per h e finire per lo, quindi $lo\$$ per dire la parola finisce per lo, e $h*$ per dire che la parola inizia per h.
- **Query con più wildcard $X*Y*Z$:** in questo caso si tratta come una ricerca per $X*Z$, per poi applicare post-filtering. Ex. se voglio cercare $h*a*o$, viene anzitutto trattata come $h*o$, quindi cerco $o\$h*$ e ottengo tutte le parole che iniziano per h e finiscono per o. Poi, tra i termini trovati, si applica un filtraggio per verificare la presenza della parte intermedia a. (es. hello fa parte di $h*o$, ma viene scartato perché non contiene a, mentre halo sarebbe stato tenuto).

Usare i permuterm porta:

- **Vantaggio:** query wildcard complicate di qualsiasi forma vengono trasformate in una semplice ricerca di prefisso nel B-tree, che è molto efficiente.
- **Svantaggio:** empiricamente, aggiungere tutte le rotazioni di ogni termine al dizionario porta a un aumento di almeno 4 volte la dimensione del dizionario (ogni termine di lunghezza n genera n+1 rotazioni).

1.2 k-grams

Un'alternativa ai permuterm è rappresentata dai k-grams, per cui invece di ruotare i termini si costruisce un secondo indice invertito che va dai k-grammi ai termini del dizionario che lo contengono.

In generale un k-gramma è una sequenza di k caratteri consecutivi in un termine. Limitiamoci a considerare i **Bigrammi**: es. per la parola hello, i bigrammi sono: \$h, he, el, ll, lo, o\$, dove il \$h e il o\$ servono per indicare l'inizio e la fine della parola. Per ogni parola del dizionario, si estraggono i bigrammi e si costruisce un indice invertito che va dai bigrammi **ai termini che li contengono** (quindi è un indice ulteriore rispetto a quello che va dai termini ai documenti, simile a quello visto prima dalle rotazioni al termine originale).

Immaginiamo di voler cercare la wildcard **mon***. Allora esistono i bigrammi \$m, mo, on, n\$, dove ad esempio \$m punta a mace, madden ..., mo punta a aMong, aMortize... on punta a alONg, amONg...

Per risolvere la query **mon***, questa può essere interpretata come \$m AND mo AND on! Questo perché usando \$m dico che la parola inizia con m, poi i bigrammi obbligatori sono mo ed on, non metto n\$ perché la query non dice che la parola deve finire con n. Quindi otterrei ad esempio money, monitor, month... **ma anche moon!** Questo perché tra i bigrammi di moon ci sono proprio \$m, mo, on → questa tecnica potrebbe restituire dei falsi positivi → **è necessario effettuare post-filtering** tra i termini ottenuti prima di effettuare la ricerca nelle postings vere e proprie.

I vantaggi rispetto al permuterm è che è un approccio space efficient se comparato a permuterm, tuttavia necessita di fare post-filtering.

Anche con i k-grammi, dopo aver filtrato i termini, bisogna eseguire query booleane sui documenti per trovare quelli che contengono ciascun termine enumerato, e quindi esattamente come per i permuterm ciò può voler dire fare merge tra liste di posting molto lunghe. Inoltre, mantenere wildcard in un motore di ricerca significa che la gente, che è pigra, tenderà ad usarle molto! Quindi in generale i grandi motori di ricerca semplicemente evitano di supportare le wildcard.

2 Spelling Correction

Lo spelling correction è un problema presente in moltissimi contesti, tra cui i motori di ricerca, in cui è necessario correggere errori di battitura o di ortografia nelle query degli utenti (nel mondo delle web queries si arriva addirittura a un tasso di errori di battitura del 26%).

Si distinguono due tipi di Spelling Tasks: **Spelling Error Detection:** (identificare se una parola è scritta in modo errato. Ad esempio, se l'utente digita "mony", il sistema dovrebbe essere in grado di riconoscere che "mony" non è una parola corretta) e **Spelling Error Correction.**

Il sistema può approcciare alla correzione in modi diversi: sostituire direttamente, suggerire una singola alternativa oppure una lista di alternative candidate. Importante puntualizzare che l'approccio aggressivo di "autocorrect" comporta un problema di **usability** dal momento che se il sistema corregge in modo errato, l'utente potrebbe non accorgersene e quindi non riuscire a trovare ciò che stava cercando. Per questo affinché si arrivi a fare autocorrect è necessario un alto livello di confidenza.

Esistono due tipi di spelling errors:

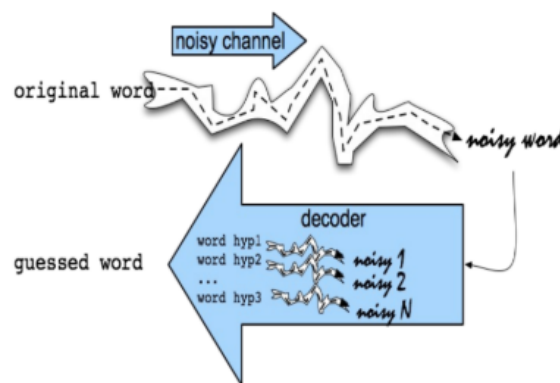
- **Non-word errors:** sono errori che producono una stringa che *non è una parola presente nel dizionario* (es. mony invece di money). Questo errore è più facile da identificare (basta controllare se la parola è presente nel dizionario) e per essere corretta è sufficiente trovare la parola più simile a quella errata (es. mony → money) → non è context sensitive.
- **Real-word errors:** in questo caso l'errore produce una stringa che *è una parola presente nel dizionario*, ma non è quella che l'utente intendeva (es. "I have a pen" invece di "I have a pan"). Più complesso da gestire, in quanto per identificarla è quasi sempre necessario analizzare il contesto in cui la parola è stata usata.

Per quanto riguarda i **Non-word errors**, come detto, la *Detection* è facile. Tuttavia bisogna fare attenzione: alcuni dizionari potrebbero contenere loro stessi dei misspellings dato che i documenti non sono necessariamente corretti ortograficamente. Per quanto riguarda invece la *Correction*: una volta capito che la parola è sbagliata, si devono generare dei candidati (parole simili all'errore) ed estrarre il migliore. Per fare ciò vedremo alcune misure: **shortest weighted edit distance** e **highest noisy channel probability**.

Quindi, per ogni parola (non necessariamente errata) w , genero un insieme di candidati cercando parole con pronuncia o grafia simile, per poi aggiungere anche w all'insieme dei candidati (potrebbe essere che w sia quella giusta). Dopodiché si sceglie il miglior candidato usando una delle misure viste prima e, nel caso di real-word errors, anche il contesto.

2.1 Noisy Channel ed Edit Distance

Noisy Channel: è un modello probabilistico che viene utilizzato per correggere errori di battitura. L'idea è che la parola corretta è stata "rovinata" da un canale rumoroso. **Conoscendo la parola sbagliata (noisy word), dobbiamo riuscire a risalire alla parola originaria, la più probabile tra tutte quelle che la potevano produrre (word hyp in figura).**



Questo concetto può essere formalizzato con Bayes: sia x la parola misspelled. Allora noi vogliamo:

$$\hat{w} = \arg \max_{w \in V} P(w|x) = \arg \max_{w \in V} \frac{P(x|w)P(w)}{P(x)}$$

Tuttavia $P(x)$ è costante per ogni candidato w , e quindi possiamo ignorarlo nella scelta del massimo. Quindi:

$$\hat{w} = \arg \max_{w \in V} P(x|w)P(w)$$

Sarebbe a dire che la parola più probabile è quella che massimizza il prodotto tra la probabilità che w sia stata trasformata in x ($P(x|w)$) e la probabilità che w sia una parola corretta ($P(w)$). Identifichiamo infatti:

- $P(w)$ come la **prior probability** di w , ossia la probabilità che w sia una parola corretta indipendentemente da x
- $P(x|w)$ come la **likelihood** di x dato w , parte integrante del noisy channel model, rappresenta la probabilità che w sia stata trasformata in x a causa del rumore (es. se $w="the"$, quanto è probabile osservare $x="teh"?$)

2.1.1 Generazione dei candidati e applicazione del noisy channel

Vediamo come funziona la generazione di candidati tramite un esempio: una query contiene la parola: "acress" (non-word error, non è presente nel dizionario). I candidati per correggere possono venire da due fonti principali: parole con *spelling simile* (quindi piccola edit distance) e parole con *pronuncia simile*. Tuttavia ora ci focalizziamo solo sulla prima categoria.

Si definisce la misura per determinare se due stringhe sono vicine: la Damerau-Levenshtein **Edit Distance**. Si tratta della distanza minima tra due stringhe quando sono consentite le seguenti operazioni elementari: *inserimento* (acress → actress), *cancellazione* (acress → acres), *sostituzione* (acress → access) e *trasposizione di due caratteri adiacenti* (hte → the, molto importante in quanto comunissimo in battitura). Qui di seguito una tabella con alcuni dei candidati a distanza 1 da acress:

Error	Candidate Correction	Correct Letter	Error Letter	Type
acress	actress	t	-	deletion
acress	cress	-	a	insertion
acress	caress	ca	ac	transposition
acress	access	c	r	substitution
acress	across	o	e	substitution
acress	acres	-	s	insertion

In generale, si usa sempre una soglia piccola sulle edit distance, dal momento che l'80% degli errori è entro distanza 1 mentre sostanzialmente tutti gli errori sono entro distanza 2. Importante aggiungere che non bisogna pensare solo a lettere singole: bisogna consentire anche inserzioni di spazio e trattini - (es. "icecream" → "ice cream", "inlaw" → "in-law") e la fusione di parole (es. data base → database).

Esistono diverse strategie per generare candidati:

1. **Scorrere tutto il dizionario:** si controlla la edit distance tra l'errore e tutti i termini del dizionario, e si selezionano quelli entro la soglia. Tuttavia è poco efficiente se il dizionario è grande.
2. **Generare le parole entro la soglia:** si producono tutte le stringhe entro distanza 1 o 2, per poi fare l'intersezione con il dizionario.
3. **Sfruttare i k-grammi:** abbiamo visto con le wildcard come si possano ad esempio salvare i bigrammi di ogni parola del dizionario in un indice dove ognuno di essi punta alla parola originale. L'idea è quindi quella di scomporre la parola della query (acress) nei vari bigrammi (\$a, ac, cr, re, es, ss, s\$) e poi cercare le parole che li contengono nel dizionario, risparmiando molto rispetto alla prima strategia. Una volta ottenuto l'elenco dei candidati, si possono determinare quelle che gli assomigliano di più ad esempio applicando la Jaccard Similarity ($|A \cap B| / |A \cup B|$) tra A = insieme di bigrammi di acress e B = insieme di bigrammi di un candidato.
4. **Usare tecniche algoritmiche come Levenshtein finite state transducer**
5. **Usare una mappa precalcolata:** una tabella già pronta con gli errori più comuni e le loro correzioni. Una mappa del genere può essere costruita manualmente oppure automaticamente, generandola dal dizionario o ancora meglio sfruttando i log degli utenti. Molto efficiente perché basta il lookup nella tabella.

Nessuna di queste tecniche è necessariamente migliore delle altre, dipende dal contesto. In pratica è troppo costoso guardare tutte le possibili correzioni e scegliere la migliore in assoluto, quindi in generale si seguono questi due passi:

- Si costruisce un insieme ristretto di candidati abbastanza buoni (es. a distanza al più 2)
- Si sceglie il migliore tra questi

Chiaro che in questo modo non è detto che la correzione scelta sia la migliore in assoluto, ma è un compromesso tra efficienza e accuratezza, e questo è un paradigma ricorrente in IR.

Ipotizziamo ora di avere generato un insieme di candidati: dobbiamo quindi calcolare $P(w|x) = P(x|w)P(w)$ per ciascuno di essi, e scegliere quello che massimizza questa quantità.

Calcolare $P(w)$: si calcola semplicemente come $C(w)/N$, dove $C(w)$ è il numero di occorrenze di w nella collezione e N è il numero totale di parole nella collezione. L'idea è che parole più frequenti sono più probabili di essere quelle corrette. In alcuni contesti, se il dizionario non basta, si possono usare anche le query digitate dagli utenti per stimare $P(w)$.

word	Frequency of word	$P(w)$
actress	9,321	.0000230573
cress	220	.0000005442
caress	686	.0000016969
access	37,038	.0000916207
across	120,844	.0002989314
acres	12,874	.0000318463

Come si osserva dalla tabella across è la parola più frequente tra i candidati. Se ci si basasse solo su $P(w)$ quindi sceglierei questa, ma non basta in quanto bisogna anche considerare $P(x|w)$ (quindi l'errore specifico).

Calcolare $P(x|w)$: non si guarda più la frequenza della parola nella collezione, ma la frequenza del tipo di errore (considerando sempre solo deletion, insertion, substitution e transposition). Ci si chiede in pratica, ad esempio, se il candidato corretto fosse actress quanto è plausibile vedere l'errore acres?

Per stimare la frequenza di questi errori si sfruttano quattro famiglie di conteggi, dove x e y sono caratteri:

- $del[x, y]$: numero di volte in cui xy è stato digitato come x
- $ins[x, y]$: numero di volte in cui x è stato digitato come xy
- $sub[x, y]$: numero di volte in cui y è stato digitato come x
- $trans[x, y]$: numero di volte in cui xy è stato digitato come yx

In particolare, per deletion e insertion, non si considera solo il carattere in questione, ma anche il carattere precedente (per questo xy e non solo x). Questo è importante perché, ipotizzando che lo user usi una tastiera, un conto è inserire subito dopo una q un w (vicine sulla tastiera, probabile che ci si sbagli!) e un conto è inserire subito dopo una q una p (non vicine). Per questo motivo, ad esempio, $del[q, w]$ sarà più alto di $del[q, p]$.

Per stimare quindi $P(x|w)$ si usano le seguenti formule:

$$P(x|w) = \begin{cases} \frac{del[w_{i-1}, w_i]}{count[w_{i-1}w_i]}, & \text{if deletion} \\ \frac{ins[w_{i-1}, x_i]}{count[w_{i-1}]}, & \text{if insertion} \\ \frac{sub[x_i, w_i]}{count[w_i]}, & \text{if substitution} \\ \frac{trans[w_i, w_{i+1}]}{count[w_iw_{i+1}]}, & \text{if transposition} \end{cases}$$

dove in generale **al numeratore si ha il conteggio del tipo di errore specifico** (nel caso di del, quante volte il bigramma $w_{i-1}w_i$ è stato digitato come solo w_{i-1} , ossia quante volte w_i è stata omessa dopo w_{i-1}) e **al denominatore il conteggio del numero totale di volte in cui è apparso il bigramma $w_{i-1}w_i$ (o un carattere nei casi di sub o ins) nella collezione**. (NB SIA NUMERATORE CHE DENOMINATORE SONO VALORI PRECOMPUTATI OFFLINE IN BENCHMARK E PRESENTI IN TABELLE DI RIFERIMENTO). In generale quindi, dato un candidato e l'errore, si considera l'edit che porta dall'errore al candidato e si calcola $P(x|w)$ usando la formula corrispondente a quel tipo di edit.

Ma come contare il numeratore, ossia il numero di volte in cui si osserva un certo tipo specifico di errore? Si sfruttano delle specifiche **Confusion Matrices**, una per ognuno dei 4 tipi di errore. Nell'immagine sotto si ha la confusion matrix per la sostituzione: in posizione (x,y) c'è il numero di volte in cui y (corretto) è stato digitato come x (sbagliato) es. (e,a) è 388 perché a è erroneamente digitato come e 388 volte. Si noti che queste matrici sono una conoscenza euristica che si basa su dati empirici.

Confusion matrix for substitution

sub(X, Y) = Substitution of X (incorrect) for Y (correct)

X	Y (correct)																									
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	0	1	0	0	0	0	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	0	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	3	3	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	0	5	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	3	0	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	14	0	2	4	14	39	0	0	0	18	0	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	3	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

Infine, è importante fare **Smoothing** (aggiungere 1 ad ogni conteggio) sulle probabilità delle matrici di confusione nel channel model. Questo è dovuto dal fatto che senza smoothing ogni errore che non è mai stato osservato nelle confusion matrix varrebbe zero → comporterebbe che la probabilità complessiva per il candidato diventerebbe zero. Facendo tutti questi ragionamenti, si arriva a stimare che il candidato più probabile per across è across.

Candidate	Correct	Error	x/w	P(x/w)	P(w)	10 ³ * P(x/w)P(w)
Correction	Letter	Letter				
actress	t	-	c c t	.000117	.0000231	2.7
cress	-	a	a #	.00000144	.000000544	.00078
caress	ca	ac	ac ca	.00000164	.00000170	.0028
access	c	r	r c	.000000209	.0000916	.019
across	o	e	e o	.0000093	.000299	2.8
acres	-	s	es e	.0000321	.0000318	1.0
acres	-	s	ss	.0000342	.0000318	1.0 ³

2.1.2 Real-word errors: caso context sensitive

Abbiamo visto come agire nel caso di non-word errors, tuttavia circa il 25-40% sono real-word errors (es. leaving in about 15 *minuets*, the study was conducted *be* John..) e in questi casi, come accennato in precedenza, per correggere correttamente necessitiamo del contesto (non posso individuarle dal dizionario perché sono parole valide!).

Come fare in modo di sfruttare il contesto per correggere la frase? Dobbiamo ragionare PER FRASE, non basta più focalizzarsi su una singola parola. In generale quindi si procede come segue:

- Per prima cosa, data la frase (query, quindi corta) $x_1, x_2, \dots, x_n$; si genera per ogni parola x_i un insieme di candidati C_i (C_i deve contenere la parola stessa (non è detto che sia un errore) e parole simili ad x_i , ad esempio ad edit distance 1),
- Si generano quindi **frasi** candidate W , e si riapplica il noisy channel model (stavolta sulla frase, non sulla singola parola!) per cercare quella che massimizza la probabilità.

Quindi si ritorna alla formula:

$$\hat{W} = \arg \max_W P(W|X) = \arg \max_W P(X|W)P(W)$$

dove però stavolta W è una frase candidata (quindi una combinazione di candidati per ciascuna parola) e X è la frase originale. $P(W)$ è la probabilità che W sia una frase corretta, e $P(X|W)$ è la probabilità che la frase corretta W sia stata trasformata in X a causa del rumore.

Ma ora, **per calcolare $P(W)$** non possiamo più applicare semplicemente $C(W)/T$ (frequenza di parole nel corpus, *Unigram Language Model*): non basta più sapere se *actress* è più o meno frequente di *across*, ma bisogna capire **quale delle due parole è più plausibile in quella posizione nella frase** (quindi, in quel contesto). Per calcolare la probabilità di una frase esistono diversi language model "contestuali", noi vediamo il più semplice: **Bigram Language Model**.

L'idea dietro il **Bigram Language Model** è quella di stimare la probabilità di una frase guardando per ogni parola solo la parola precedente:

$$P(W) = P(w_1, w_2, \dots, w_n) \approx P(w_1)P(w_2|w_1)P(w_3|w_2) \dots P(w_n|w_{n-1})$$

In questo modo, tornando all'esempio di *actress* ed *across*, il modello permette di determinare che la frase "versatile actress whose" è più probabile di "versatile across whose", in quanto le coppie "versatile actress" e "actress whose" sono più plausibili rispetto a "versatile across" e "across whose".

In generale, per calcolare $P(w_k|w_{k-1})$ si usa la formula:

$$P(w_k|w_{k-1}) = \frac{C(w_{k-1}, w_k)}{C(w_{k-1})}$$

dove $C(w_{k-1}, w_k)$ è il numero di volte in cui il bigramma $w_{k-1}w_k$ è apparso nella collezione, e $C(w_{k-1})$ è il numero di volte in cui la parola w_{k-1} è apparsa nella collezione. Chiaro che per poter calcolare questo valore si devono mantenere in una struttura dati (ad esempio lo stesso inverted index) anche i bigrammi e le loro occorrenze nel corpus.

Sorge un problema: mentre nel caso degli unigrammi la probabilità $P(w)$ è sempre maggiore di zero (ogni parola ha una frequenza), nel caso dei bigrammi è molto comune che $P(w_k|w_{k-1})$ sia zero, in quanto è possibile che la coppia $w_{k-1}w_k$ non sia mai apparsa nella collezione. Per risolvere il problema si può agire in due modi:

- **Smoothing**: esattamente come visto prima, si aggiunge 1 a ogni conteggio, in modo da evitare che la probabilità di un bigramma sia zero.
- **Interpolazione unigram + bigram**: invece di "fidarsi" solo del bigramma si può considerare anche la probabilità del singolo unigramma nel conteggio, facendo la media pesata tra i due:

$$P(w_k|w_{k-1}) = \lambda P(w_k) + (1 - \lambda)P(w_k|w_{k-1})$$

In questo modo, anche se il bigramma $w_{k-1}w_k$ non è mai apparso nella collezione, si tiene conto della frequenza di w_k come unigramma e ci si assicura di non avere probabilità zero. Inoltre se w_k è molto frequente, si dà la possibilità al bigramma nella query di essere scelto anche se non è apparso nella collezione, rendendo il tutto più flessibile.

Per quanto riguarda invece il calcolo di $P(X|W)$, si applica sempre il noisy channel model, ma generalizzato a livello di frase.

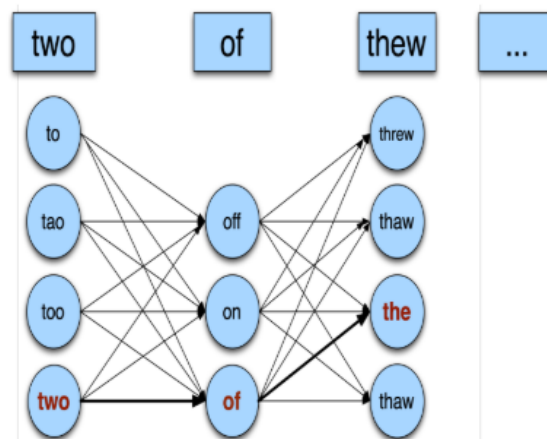
Qui sotto un esempio del calcolo della $P(W)$, dimostrando come si possa riconoscere *actress* invece di *across* sfruttando il contesto nonostante come visto prima *across* sia un unigramma più frequente di *actress*:

- "a stellar and versatile **acress** whose combination of sass and glamour..."
- Counts from the Corpus of Contemporary American English with add-1 smoothing
- $P(\text{actress}|\text{versatile}) = .000021$ $P(\text{whose}|\text{actress}) = .0010$
- $P(\text{across}|\text{versatile}) = .000021$ $P(\text{whose}|\text{across}) = .000006$
- $P(\text{"versatile actress whose"}) = .000021 * .0010 = 210 \times 10^{-10}$
- $P(\text{"versatile across whose"}) = .000021 * .000006 = 1 \times 10^{-10}$

Dal momento che per calcolare $P(W)$ è necessario fare diverse moltiplicazioni tra probabilità molto piccole, si arriva a valori minuscoli. Ciò è problematico a livello pratico in quanto può portare i computer ad andare in underflow e non riuscire a rappresentarli → si lavora con i logaritmi (mantengono le proporzioni, ma invece del prodotto si ha la somma e si evita il problema):

$$\log(P(W)) = \log(P(w_1)) + \log(P(w_2|w_1)) + \dots + \log(P(w_n|w_{n-1}))$$

Un punto importante è che nel calcolo di $P(W)$ è il fattore $P(w_1)$. Dato che infatti la prima parola non ne ha una precedente la si valuta come unigramma. In realtà, nei language models classici, può essere utile sapere che w_1 è la prima parola di una frase e quindi si introduce un simbolo speciale (tipo $\langle s \rangle$) per indicarlo e si scrive qualcosa del tipo $P(w_1|\langle s \rangle)$. *Tuttavia nel caso delle query di ricerca, tipicamente gli utenti digitano frasi incomplete, e quindi non è detto che la prima parola sia effettivamente la prima parola di una frase. Per questo motivo ha senso trattare $P(w_1)$ come un unigramma, senza introdurre $\langle s \rangle$.*



Come si osserva dall'immagine sopra quindi, per ogni parola della frase si generano dei candidati, e il sistema deve riuscire a scegliere un cammino attraverso di essi che formi la sequenza più plausibile. *Calcolare questa sequenza come visto prevede sia l'uso di language model (per calcolare $P(W)$) che del noisy channel model (per calcolare $P(X|W)$).* Per trovare il cammino ottimo tra i candidati, si può usare un algoritmo di programmazione dinamica chiamato **Algoritmo di Viterbi**. (Quindi in pratica invece di calcolare tutte le possibili combinazioni di candidati (quindi tutte le possibili frasi) e fare la probabilità con il noisy channel come visto prima, si usa un algoritmo di programmazione dinamica che permette di trovare la sequenza più probabile in modo efficiente).

Domanda (al di là di Viterbi che aiuta a trovare il cammino ottimo senza bruteforce, si può semplificare il problema): abbiamo detto che per ogni parola della frase si generano dei candidati, da cui si ha una frase W per ogni possibile combinazione e si calcola $P(W)$ per ciascuna di esse. Ma se ogni parola ha un grande numero di candidati, il numero di W esplode! Ma nella realtà tipicamente, un utente fa in media al massimo 1-2 errori per query, quindi si assume che **per ogni frase ci sia solo un errore**, riducendo di molto il numero di combinazioni da valutare.

Importante: come già accennato, nel channel model (per calcolare $P(X|W)$ nel caso real word o $P(x|w)$ nel caso non-word) ci sono oltre alle probabilità di errore come sostituzione, cancellazione, inserzione e trasposizione, **anche la probabilità di nessun errore $P(w|w)$!** Nel caso dei non-word errors, dal momento che la parola viene identificata in quanto non presente nel dizionario, il sistema

- Out of all possible sentences with one word replaced
 - w_1, w''_2, w_3, w_4 two off thew
 - w_1, w_2, w'_3, w_4 two of the
 - w'''_1, w_2, w_3, w_4 too of thew
 - ...
- Choose the sequence W that maximizes $P(W)$

parte con il presupposto che la parola sia sbagliata, quindi l'impatto di questa probabilità è minimo. Nel caso invece dei **real-word errors**, dal momento che le parole osservate nella frase sono tutte valide, si deve prendere in considerazione la possibilità che la parola osservata sia effettivamente corretta.

Come stimare $P(w|w)$? La stima è molto empirica e dipende dall'applicazione specifica: si può ad esempio guardare la frequenza con cui la parola w è stata digitata correttamente dagli utenti, se su 10 volte che w è stato scritto, una volta è stata sbagliata in media allora potremmo dire che $P(w|w) = 0.9$, se in media invece gli utenti la sbagliano una volta su 100 $P(w|w) = 0.99$, e così via. Questa probabilità è particolarmente importante per la questione di usability accennata in precedenza: il modello deve preferire non correggere se non è "sufficientemente sicuro".

Problema:

Peter Norvig's "thew" example

x	w	x w	P(x w)	P(w)	$10^9 P(x w)P(w)$
thew	the	ew e	0.000007	0.02	144
thew	thew		0.95	0.00000009	90
thew	thaw	e a	0.001	0.0000007	0.7
thew	threw	h hr	0.000008	0.000004	0.03
thew	thwe	ew w	0.000003	0.00000004	0.0001

Si vede nell'immagine sopra il Norvig's "thew" example (qui per semplicità ipotizziamo che thew venga trattata come parola isolata, nonostante sia una real word e quindi si dovrebbe applicare il bigram model). Nella query viene letto "thew", che è una parola corretta. Il sistema inizia quindi a ragionare: genera i candidati (tra cui *the*) e calcola $P(w|x)P(w)$ per ciascuno di essi. Qui sorge il problema: **dato che *the* ha una frequenza altissima**, $P(the)$ è molto alto e riesce anche a compensare $P(thew|the)$ che invece è molto basso (per via della deletion di w), a tal punto che il loro prodotto supera quello dello stesso thew ($P(thew|thew) \cdot P(thew)$), che era però la parola corretta!

Per questa ragione calcolare $P(x|w) \cdot P(w)$ non è sufficiente, in quanto $P(x|w)$ e $P(w)$ sono probabilità che si basano su assunzioni diverse. La soluzione quindi quella di **introdurre un peso λ per attenuare o rafforzare la prior probability $P(w)$** . Di seguito quindi la formula finale, state of art, per noisy channel model:

$$\hat{w} = \arg \max_{w \in V} P(x|w)P(w)^\lambda$$

Intuitivamente, all'aumentare di λ si dà più peso alla lingua (e quindi alla probabilità che la parola appaia nella collezione), mentre se λ è più piccolo allora si dà maggiore peso al channel model e quindi alla possibilità di errori. Nel caso thew di prima, è chiaro che sarebbe servito un λ più piccolo per smorzare l'effetto di $P(the)$ e quindi lasciare correttamente thew.

Come determinare λ ? Empiricamente, sul development set, provando diversi valori e scegliendo quello che massimizza la performance. (quindi si prende un "piccolo" set di esempi per cui si sapeva la correzione giusta, si fa parameter tuning sul lambda e si sceglie quello che massimizza la performance).

Improvements to channel model: il modello di errore può essere ulteriormente migliorato, ad esempio:

- Con edit più complessi (non solo inserzione, cancellazione etc... ma anche edit più realistici, legati anche alla fonetica, es. ent $\rightarrow$ ant, ph $\rightarrow$ f etc..)

- Aggiungere attenzione agli errori fonetici
- Distinguere per dispositivi (es. errori da mobile vs errori da desktop)